Regression Models

> Part of the Data Science / ML curriculum. This document covers Naive baseline, Linear Regression, and Random Forest — the three classical regression models most commonly used for tabular/panel prediction problems — the concepts behind each, the scikit-learn API, the parameters worth understanding, and how to evaluate and compare them correctly.

1. The modeling task

Regression means predicting a continuous numeric quantity (as opposed to **classification**, which predicts a discrete category). Given a table of features `X` (the engineered columns from Feature Engineering) and a known target `y` for historical rows, the goal is to learn a function `f(X) ≈ y` that generalizes to rows where `y` isn't known yet.

The models below are presented in increasing order of complexity, which is also usually the order you'd try them in on a new problem: establish a floor (baseline), try a simple model, then a more flexible one — and compare all of them on the same held-out data (§6).

2. Naive baseline

The simplest possible forecast: predict that the next period's value equals the current period's value (a "persistence" forecast). There's no library call for this — it's just:

```
y_pred_naive = df["value_lag1"]   # the lag-1 feature from Feature Engineering, §2.3
```

Its purpose isn't to be good; it's to be a **floor**. Any real model that can't beat a naive baseline isn't adding value, and reporting a naive baseline alongside every other model is the only reliable way to know whether that's happening. A model that looks impressive in isolation (a low absolute error) can still be worthless if the naive baseline achieves the same error for free.

A closely related variant for data with a repeating seasonal pattern is the **seasonal naive** forecast: predict this period's value equals the value from the same point in the previous cycle (e.g., last January for this January) — a stronger baseline than plain persistence whenever seasonality is present, though not relevant for the annual, non-seasonal data this series focuses on (see Time-Series Forecasting, §2 for where seasonality does and doesn't apply).

3. Linear Regression

3.1 Concept

Fit a linear function of the input features to the target — find coefficients $\beta_0, \beta_1, \ldots, \beta_n$ such that $\text{target} \approx \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \ldots + \beta_n \cdot x_n$, minimizing the sum of squared errors (**ordinary least squares**, OLS). It's the right first real model to try because it's fast to fit, has very few knobs to get wrong, and its coefficients are directly interpretable — the sign and magnitude of each β tells you how that feature relates to the target, holding the others constant.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)          # X_train: 2D array/DataFrame of features, y_train: 1D target
predictions = model.predict(X_test)

model.coef_        # one coefficient per feature, in the same order as X_train's columns
model.intercept_   # the β₀ term
```

Key parameter (`sklearn.linear_model.LinearRegression`): `fit_intercept` — whether to fit a β_0 term at all (leave `True` unless you have a specific reason your target must pass through the origin when every feature is zero).

3.2 Assumptions worth knowing

Ordinary least squares linear regression formally assumes:

- **Linearity** — the true relationship between features and target is (approximately) linear.
- **Independence** — observations are independent of each other (a real limitation for panel/time-series data, where consecutive observations for the same entity are correlated by construction — this is one reason a purely cross-sectional linear model is often supplemented with lag features, §2.3 of Feature Engineering, to at least partially capture that dependence within the feature set itself).
- **Homoscedasticity** — the variance of the errors is roughly constant across the range of predictions (not, e.g., much larger for high-value predictions than low-value ones).
- **Normally-distributed residuals** — mainly matters for the validity of statistical significance tests on the coefficients, less so for pure predictive accuracy.

Violating these assumptions doesn't necessarily make the model useless, but it does mean the model's own internal confidence (e.g., p-values on coefficients, if you're using a library that reports them) may be unreliable — predictive accuracy on a genuine holdout set (§6) is a more robust way to judge the model regardless of how well these assumptions hold.

3.3 Regularized variants

Plain `LinearRegression` can overfit when features are highly correlated with each other, or when there are many features relative to the number of rows. Adding a **regularization** penalty discourages the model from assigning very large coefficients:

```python
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)   # alpha: strength of the penalty — higher alpha = more shrinkage toward zero
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=1.0)   # like Ridge, but can shrink some coefficients exactly to zero
lasso.fit(X_train, y_train)
```

`Ridge` (L2 penalty) shrinks all coefficients toward zero without necessarily eliminating any; `Lasso` (L1 penalty) can shrink some coefficients to *exactly* zero, effectively performing feature selection as a side effect. `alpha` is the key parameter for both — usually chosen via cross-validation (see `RidgeCV` / `LassoCV`, which search over a range of `alpha` values automatically) rather than picked by hand.

3.4 An alternative: statsmodels for interpretability

`scikit-learn`'s `LinearRegression` is optimized for prediction; if you want p-values, confidence intervals, and a full statistical summary of the fit, **statsmodels**' `OLS` is the more common tool:

```python
import statsmodels.api as sm

X_train_with_const = sm.add_constant(X_train)   # statsmodels doesn't add an intercept automatically
model = sm.OLS(y_train, X_train_with_const).fit()
print(model.summary())   # coefficients, standard errors, p-values, R², and more
```

3.5 A key caveat: unbounded extrapolation

Linear Regression extrapolates linearly forever — nothing stops a prediction from going arbitrarily far outside the range of training data, including past zero into physically impossible values for a quantity that can't be negative. This matters a great deal for multi-step recursive forecasting — see Time-Series Forecasting, §5.

4. Random Forest

4.1 Concept

An **ensemble** of many decision trees, each trained on a bootstrapped (randomly resampled, with replacement) subset of the training rows, with each tree's splits also restricted to a random subset of features at each split point. The forest's prediction is the average of all its trees' predictions. This combination — bagging (bootstrap aggregating) plus feature randomness — reduces the variance any single decision tree would have on its own, and lets the model capture non-linear relationships and interactions between features that a linear model can't represent.

```python
from sklearn.ensemble import RandomForestRegressor


model = RandomForestRegressor(
    n_estimators=100,      # number of trees — more is generally better but with diminishing returns and rising cost
    max_depth=None,        # unlimited by default; set an int to limit tree depth and reduce overfitting on small data
    min_samples_leaf=1,    # minimum samples required at a leaf node — raise this to smooth predictions on noisy/small data
    max_features="sqrt",   # number of features considered at each split — "sqrt" (√ of total features) is a common default for regression
    bootstrap=True,        # whether each tree trains on a bootstrapped sample (True) or the full dataset (False)
    oob_score=False,       # if True, uses the ~37% of rows NOT in a given tree's bootstrap sample to estimate accuracy "for free", without a separate holdout
    n_jobs=-1,             # number of CPU cores to use in parallel; -1 uses all available cores
    random_state=42,       # fixes the randomness (bootstrap sampling, feature selection) for reproducible results
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

model.feature_importances_   # relative importance of each feature (mean decrease in impurity), same order as X_train's columns
```

4.2 Feature importance, and a caveat about it

`feature_importances_` (mean decrease in impurity) is convenient but has a known bias: it tends to inflate the importance of high-cardinality numeric features, even when they're not genuinely more predictive. **Permutation importance** (`sklearn.inspection.permutation_importance`) is a more reliable, model-agnostic alternative: it measures how much a model's accuracy *drops* when a single feature's values are randomly shuffled (destroying that feature's relationship with the target) — a feature the model genuinely relies on will cause a large accuracy drop when shuffled; a feature it ignores won't.

```python
from sklearn.inspection import permutation_importance


result = permutation_importance(model, X_test, y_test, n_repeats=10, random_state=42)
result.importances_mean   # same order as X_test's columns
```

4.3 The critical trade-off: model complexity vs. data availability

A Random Forest with `n_estimators=100` has real capacity — and capacity needs data to be used well. Trained on very few rows (tens, not hundreds), a 100-tree ensemble will effectively memorize the training set's noise rather than learn a generalizable pattern, and can easily underperform even the naive baseline (§2) on unseen data. This isn't a bug in the algorithm; it's a mismatch between model complexity and data volume — a specific instance of the more general **bias-variance trade-off** (§5).

The practical fix, when your data is naturally segmented (e.g., one small time series per entity) but each individual segment doesn't have enough rows on its own, is **pooling**: combine data across segments into one larger training set, and add a categorical feature identifying which segment each row came from (see Feature Engineering, §5.1), so the model can still distinguish between segments while learning shared patterns across all of them.

4.4 Hyperparameter tuning

Rather than guessing values for `n_estimators`, `max_depth`, etc. by hand, search over a range of candidates and pick the combination that performs best on validation data:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_leaf": [1, 5, 10],
}
search = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid,
    scoring="neg_mean_absolute_error",   # scikit-learn convention: scores are maximized, so error
        metrics are negated
    cv=3,                                  # see §6.2 for why this needs care on time-series data
)
search.fit(X_train, y_train)
search.best_params_    # the winning combination
search.best_estimator_ # a model already refit on the full training set with those parameters
```

`GridSearchCV` tries every combination in `param_grid` exhaustively; `RandomizedSearchCV` samples a fixed number of random combinations instead, which scales better when the grid is large. Both rely on cross-validation internally to score each candidate — see §6.2 for why the default cross-validation strategy needs to be replaced for time-dependent data.

5. The bias-variance trade-off

This is the single most important conceptual idea tying every model above together:

- **Bias** is error from a model being too simple to capture the true pattern (Linear Regression forced onto a genuinely non-linear relationship; the naive baseline ignoring everything except the most recent value). A high-bias model tends to *underfit* — it performs similarly poorly on both training and test data, because it simply isn't flexible enough to represent the pattern at all.
- **Variance** is error from a model being so flexible that it fits noise specific to the training sample, rather than the true underlying pattern (a very deep, unconstrained Random Forest trained on very few rows). A high-variance model tends to *overfit* — it performs very well on training data but much worse on unseen test data, because what it "learned" partly describes noise that won't repeat.

The practical implication: **model complexity must match data availability.** A simple model trained on data adequate for its complexity typically outperforms a complex model trained on insufficient data — this is exactly what §4.3's pooling discussion is addressing. Watching the gap between training-set performance and test-set performance is the standard diagnostic: a small gap with both scores poor suggests underfitting (try a more flexible model, or better features); a large gap (great on training, much worse on test) suggests overfitting (simplify the model, get more data, or add regularization).

6. Evaluating models the right way

6.1 Train/test splitting for time-dependent data

Do not use a random shuffled split for time-series data. Scikit-learn's `train_test_split(shuffle=True)` (the default) randomly assigns rows to train/test, which for time-ordered data means the model can end up training on rows from *later* in time than some of its test rows — effectively letting it see the future during training. This produces metrics that look better than the model will actually perform in real use.

```
train = df[df["year"] <= CUTOFF_YEAR]
test  = df[df["year"] >  CUTOFF_YEAR]
```

A well-chosen test window should include at least one genuine "stress test" if the data has one available — a period with an unusual shock (a recession, a crisis, any discontinuity) — since a model's behavior during an out-of-distribution event is often the most informative thing you can learn about it.

6.2 Cross-validation — and why the standard version doesn't apply here

K-Fold cross-validation (scikit-learn's default `cv` behavior) splits data into *K* random folds, trains on *K-1* of them, and validates on the remaining one, rotating which fold is held out — this gives a more robust performance estimate than a single train/test split by averaging over several splits. It assumes rows are **exchangeable** (i.i.d. — the order they appear in doesn't matter), which is exactly the assumption that random shuffling violates for time-series data (§6.1).

For time-dependent data, use **`TimeSeriesSplit`** instead, which respects chronological order — every validation fold comes strictly after the training data used to predict it:

```
from sklearn.model_selection import TimeSeriesSplit, cross_val_score

tscv = TimeSeriesSplit(n_splits=5)
scores = cross_val_score(model, X, y, cv=tscv, scoring="neg_mean_absolute_error")
```

Use `TimeSeriesSplit` (not plain `KFold`) as the `cv=` argument anywhere in this document a cross-validation strategy is needed for time-ordered data — including inside `GridSearchCV` / `RandomizedSearchCV` (§4.4).

6.3 Metrics

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae  = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2   = r2_score(y_true, y_pred)
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100   # careful: undefined if any y_true == 0
```

- **MAE (Mean Absolute Error)** — average absolute difference between predicted and actual values, in the same units as the target. Easy to interpret directly ("on average, off by X").
- **RMSE (Root Mean Squared Error)** — similar, but squares errors before averaging (then square-roots the result), which penalizes large errors disproportionately more than small ones. RMSE ≥ MAE always; the gap between them is itself informative — a much larger RMSE than MAE means a few large errors are dragging the average up (inconsistent performance), rather than uniformly-mediocre performance.
- **R^2 (coefficient of determination)** — the fraction of variance in the target explained by the model, on a scale where 1.0 is a perfect fit and 0.0 means "no better than always predicting the mean" (it can go negative for a model that's worse than that). Useful as a normalized, scale-free summary, but less intuitive than MAE/RMSE for communicating "how wrong is a typical prediction" in real units.
- **MAPE (Mean Absolute Percentage Error)** — average absolute error as a percentage of the true value, useful for comparing error magnitude across series of very different scale — but undefined (division by zero) whenever the true value is zero, and can be disproportionately dominated by rows where the true value happens to be very small.

Compute metrics per segment (e.g., per entity) when your data is naturally segmented — a single pooled metric across very different segments can hide a model that works well for some and poorly for others.

6.4 Residual analysis

Beyond a single summary metric, plot the **residuals** (actual minus predicted) against the predicted value, and against time:

```python
residuals = y_test - predictions
plt.scatter(predictions, residuals)
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Predicted"); plt.ylabel("Residual")
```

A well-fitting model's residuals should look like structureless noise scattered around zero. A visible pattern — residuals trending up or down over time, or fanning out as predictions get larger (heteroscedasticity) — indicates the model is systematically missing something a single summary metric wouldn't reveal on its own.

7. Other models worth knowing about (brief)

The three models above cover the core of this curriculum, but two related model families are worth knowing exist, since you'll encounter them in most real-world tabular ML work:

- **Gradient boosting** (`GradientBoostingRegressor` in scikit-learn; the widely-used external libraries **XGBoost**, **LightGBM**, and **CatBoost**) builds an ensemble of trees *sequentially*, where each new tree is trained to correct the errors of the ones before it, rather than averaging independently-trained trees the way Random Forest does. This frequently achieves better accuracy than Random Forest on tabular data, at the cost of more hyperparameters to tune carefully (it's more prone to overfitting if not tuned well) and slower, sequential (less parallelizable) training.
- **Regularized linear models** beyond Ridge/Lasso (§3.3) — **ElasticNet** (a Ridge+Lasso combination) and generalized linear models for non-Gaussian targets (e.g., Poisson regression for count data) — are worth knowing exist if your target variable doesn't fit the assumptions in §3.2 well.

8. Core libraries summary

Library	Used for
scikit-learn	`LinearRegression`, `Ridge`, `Lasso`, `RandomForestRegressor`, `TimeSeriesSplit`, `GridSearchCV`, metrics (`mean_absolute_error`, `mean_squared_error`, `r2_score`), `permutation_importance`
statsmodels	`OLS` for a fuller statistical summary of a linear fit
NumPy	underlying numeric operations, manual metric calculations
Matplotlib	residual plots and other diagnostic charts

9. Testing and validating model code

- **The chronological train/test split (§6.1) is itself the core validation methodology** — it directly measures how the model would have performed on data it hasn't seen, which is the property that actually matters.
- **Regression-test your metrics once you have a stable pipeline**: after a model's training/evaluation logic is extracted into reusable functions, a `pytest` test can assert that a known small dataset produces a metric within an expected range — this catches accidental regressions (a refactor that subtly changes what data reaches the model) even without re-validating the model's actual predictive quality on every commit.
- **Deliberately break an assumption to confirm your evaluation catches it** — e.g., temporarily use `train_test_split(shuffle=True)` instead of a chronological split, and confirm the reported error metric changes noticeably. If it doesn't change at all, that's a sign your evaluation wasn't sensitive to the leakage in the first place.

See also

- Data Science / ML index
- Feature Engineering — the previous step, building the inputs these models consume
- Time-Series Forecasting — a different family of models for the same kind of data, built directly on the target series itself rather than engineered features